47. Git on the Server and Hooks (ops angle)

47. Git on the Server and Hooks (ops angle)

Phase: Phase 5 — Shell scripting
Estimated time: 1 day
Slug: `47-git-on-linux`

Learning objectives

- Use git safely on servers
- Understand deploy keys
- Avoid editing prod checkouts casually
- Know hook basics

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Git is how code reaches servers, but "git on the server" can mean wildly different things. At one extreme, an engineer SSHs into production, runs `git pull` in `/opt/app`, edits a config file in place, and leaves the checkout dirty. Six months later nobody knows what is actually running. At the other extreme, CI builds a versioned artifact once, and servers receive that exact artifact and nothing else. The second approach makes rollbacks trivial and incidents explainable.

Prefer an immutable artifact: a tarball, package, or container image built once from a tag, checksummed, and deployed unchanged. Git remains the source of truth for code, while the artifact is the unit of deployment.

Core concepts

Concept	Meaning
Deploy key	SSH key with read-only access to one repo
Tag	immutable name for a commit, such as `v1.4.2`
Artifact	built output, versioned and checksummed
Dirty tree	local modifications that differ from the commit
Hook	script git runs on events (`pre-commit`, `post-receive`)
Bare repo	repo without a working tree, used as a remote
Rollback	redeploy previous artifact by version

```
  dev --> git push --> CI builds --> artifact app-1.4.2.tgz (sha256)
                                          |
                        registry / artifact store
                                          |
          server: fetch version -> verify checksum -> unpack to
          /opt/app/releases/1.4.2 -> symlink current -> restart
          rollback = point `current` at releases/1.4.1
```

Worked examples

Example 1: check what a server checkout really is.

```
cd /opt/app
git status --short
git describe --tags --always
git diff --stat
```

Any output from `git status --short` means the running code differs from any commit. That is a finding worth recording.

Example 2: read-only deploy key.

```
ssh-keygen -t ed25519 -f ~/.ssh/deploy_app -C "deploy-app-ro" -N ""
# add the .pub as a read-only deploy key in the repo settings
GIT_SSH_COMMAND="ssh -i ~/.ssh/deploy_app -o IdentitiesOnly=yes" \
  git ls-remote git@example.com:org/app.git
```

Example 3: release directories with an atomic symlink switch.

```
REL=/opt/app/releases/1.4.2
sudo mkdir -p "$REL"
sudo tar -xzf app-1.4.2.tgz -C "$REL"
sudo ln -sfn "$REL" /opt/app/current
sudo systemctl restart app.service
```

Example 4: verify before deploying.

```
sha256sum -c app-1.4.2.tgz.sha256
git verify-tag v1.4.2
```

Example 5: a minimal server-side hook in a bare repo, used only to trigger CI or notify, not to hot-edit production.

```
#!/usr/bin/env bash
# hooks/post-receive
while read old new ref; do
  echo "$(date -Is) $ref $new" >> /var/log/git-push.log
done
```

Mini-lab

1. Create a repo in `/tmp/lab/app`, commit a file, and tag it `v1`.
2. Build an artifact with `git archive --format=tar.gz -o app-v1.tgz v1`.

3. Compute `sha256sum app-v1.tgz` and save it.
4. Unpack into `releases/v1`, link `current`, and list the layout.
5. Make a second tag `v2`, repeat, switch the symlink, then switch it back.
6. In `releases/v2`, notice there is no `.git` directory; explain why that is an advantage on a server.

Decision guide / failure modes

Situation	Prefer
Production deploy	immutable artifact from a tag
Server needs source for debugging	read-only clone in a separate path
Secrets in the repo	remove, rotate, use a secret store
Need to pull from private repo	deploy key, read-only, one per repo
Hotfix at 3am	still build an artifact, even if the pipeline is shortened

Failure modes: using `git pull` on a branch (not a tag) so the version moves without notice; storing a personal SSH key on a server; write-enabled deploy keys; editing files in the checkout; deploying `.git` and secrets into a web-served directory; and not recording the version that is live. Keep a `VERSION` file or use `git describe` output baked into the artifact so `curl /version` answers "what is running".

Common mistakes

- git pull as root on live main
- Write-capable deploy keys broadly
- Uncommitted hotfixes on servers

Exercises

Work in `exercises/47-git-on-linux/`. Read that folder's `README.md` fully before coding.

```
cd exercises/47-git-on-linux
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `prefer_deploy_artifact`. It prints a short hyphenated token that names the kind of deployable unit the lesson recommends over pulling a mutable branch on a server. Lowercase, one line, nothing else. After it passes, do the mini-lab and capture the output of `ls -l /tmp/lab/app/releases` and the `sha256sum` line. In notes, write the rollback steps as exactly two commands, and explain why a dirty `git status --short` on a production host is an incident finding, not a normal state.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/47-git-on-linux/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Why immutable artifacts?
2. Deploy key purpose?
3. Risk of prod git pull?

Further reading

- https://git-scm.com/book/en/v2